**Listing 12.8 The inner template for the search results**

```
<!-- RiffMates/templates/partials/musician_results.html -->
<ul>
  {% for musician in musicians %}
    <li> <a href="{% url 'musician' musician.id %}">
      {{musician.last_name}}, {{musician.first_name}}
    </a> </li>
  {% empty %}
    <li> <i>No matching musicians</i> </li>
  {% endfor %}
</ul>
```

Loop through and display all the musicians.

An HTMX-enabled search page isn't very different from a regular one; with a few extra attributes, you have search-as-you-type. The only thing missing now is a view that returns the results as partial HTML. In the `promoters()` view early in this chapter, you used separate views for the main page and the partial, but like with GET and POST sections in a form view, it is cleaner to put it all in the same place.

For a view to be able to render both the page and the partial segment, it needs to know its invocation mode. Any call that HTMX makes includes a header called `HX-Request`. In your view, you check for this header and behave accordingly. You could do this manually, but there is a very handy library that makes it simpler: django-htmx. To install this library, call `python -m pip install django-htmx`, add `"django_htmx"` to your `INSTALLED_APPS`, and add `"django_htmx.middleware.HtmxMiddleware"` to the `MIDDLEWARE` list in your settings.py file. For more details on installing third-party libraries, see appendix A.

The `HtmxMiddleware` component of `django-htmx` adds an `htmx` object to the `request` in your view. This object contains details about the HTMX call and, usefully, can be used in a Boolean context to determine whether the HTMX invoked the call. The following listing contains the new version of the search musicians view.

**Listing 12.9 An HTMX-capable search view**

```
# RiffMates/bands/views.py
...
def search_musicians(request):
    search_text = request.GET.get("search_text", "")
    search_text = urllib.parse.unquote(search_text)
    search_text = search_text.strip()

    musicians = []

    if search_text:
        parts = search_text.split()

        q = Q(first_name__istartswith=parts[0]) | \
            Q(last_name__istartswith=parts[0])
        for part in parts[1:]:
            q |= Q(first_name__istartswith=part) | \
                 Q(last_name__istartswith=part)
```